VELORA — PERMANENT SECURITY AUDIT CHECKLIST

SECURITY BASELINE FOR EVERY VERSION

Purpose

This is the permanent security checklist for the Velora project.

Run this security audit:

- before every production release
- after every major feature
- after backend/API changes
- after authentication changes
- after database changes
- after file-upload changes
- after payment-related changes
- after AI integration changes
- after infrastructure/server changes

The purpose is to detect:

1. New vulnerabilities
2. Existing vulnerabilities
3. Security regressions
4. Information leakage
5. Misconfigurations
6. Exposed sensitive files
7. Broken access controls
8. Unsafe APIs
9. Unsafe uploads
10. Authentication problems

Audit-only rule

This is an audit only.

During the audit:

- Do not modify the codebase.
- Do not automatically fix vulnerabilities.
- Do not delete files.
- Do not change configuration.
- Do not rotate credentials.
- Do not modify the database.

If a vulnerability is found, report it first with:

- severity
- affected component
- evidence
- risk
- recommended fix
- verification test

Wait for approval before making security changes.

---

1. Sensitive File Exposure

Check whether publicly accessible URLs expose:

- `.env`
- `.env.local`
- `.env.production`
- `.env.development`
- `.git/`, `.git/config`, `.git/HEAD`
- inappropriate package/composer lock or manifest files
- database dumps and SQL files
- backup, ZIP, TAR, and old deployment files
- sensitive source maps
- debug and temporary files
- server/PHP/application configuration
- application/error logs
- test files containing secrets

The following must not be publicly accessible:

- `/.env`
- `/api/.env`
- `/.git/`
- `/backup/`
- `/backups/`
- `/logs/`
- `/config/`
- `/database/`
- `/private/`

Test appropriate variations such as:

- `/.env`
- `/api/.env`
- `/.git/config`
- `/config.php`

- `/database.sql`
- `/backup.zip`

Expected result: `404`, `403`, or another secure denial. Sensitive contents must never be returned.

2. Environment Variables and Secrets

Check API keys, database credentials, JWT/session secrets, encryption keys, AI/payment/SMTP/cloud credentials, and webhook secrets.

Verify that secrets:

- remain server-side
- are not hardcoded
- are not exposed to the frontend
- are not included in Git or JavaScript bundles
- are not returned by APIs
- are not written to logs

Never print real secret values. If a value must be identified, mask it.

3. Git and Source-Code Exposure

Check production exposure of:

- `.git` and repository metadata
- source/development configuration
- debug scripts and test endpoints
- internal documentation
- deployment scripts

Production must not expose unnecessary source or operational files.

4. Authentication

Audit login, logout, registration, password reset, email verification, sessions, tokens, remember-me, expiration, and brute-force protection.

Check for weak authentication, predictable tokens, missing expiration, insecure cookies, session fixation, account enumeration, and brute-force weaknesses.

5. Password Security

Verify:

- modern password hashing
- no plaintext password storage
- no password logging
- secure reset tokens
- reset-token expiration
- reset-token single use

6. Authorization and Access Control

Verify users cannot access another user's:

- trades and journal entries
- screenshots and attachments
- strategies, notes, and tags
- account and profile information

Perform safe IDOR tests. User B must receive `403`, `404`, or access denial for User A's private resource.

7. API Security

For every endpoint document:

- authentication requirement
- authorization requirement
- input validation
- rate limiting
- sensitive response fields
- error leakage
- allowed HTTP methods

Pay special attention to `POST`, `PUT`, `PATCH`, and `DELETE`. Users must not modify resources they do not own.

8. Input Validation

Check all user-controlled input, including symbol, prices, volume, notes, tags, search, filters, IDs, dates, filenames, URLs, and query parameters.

Server-side validation is mandatory; frontend-only validation is insufficient.

9. SQL Injection

Audit database queries for unsafe concatenation. Verify prepared/parameterized queries, ORM safety where applicable, and constrained dynamic SQL.

Testing must be non-destructive.

10. XSS

Check stored, reflected, and DOM XSS, especially in trade/journal notes, strategy names, tags, profile fields, blog content, and search.

User-controlled content must not execute as HTML or JavaScript.

11. CSRF

Audit state-changing requests and evaluate CSRF tokens where needed, SameSite cookies, Origin validation, and the authentication architecture, especially for `POST`, `PUT`, `PATCH`, and `DELETE`.

12. CORS

Verify production does not unnecessarily allow `Access-Control-Allow-Origin: *`, especially for authenticated APIs. Allow only trusted origins where appropriate.

13. Security Headers

Evaluate:

- `Content-Security-Policy`
- `X-Content-Type-Options`
- `Referrer-Policy`
- `Permissions-Policy`
- `Strict-Transport-Security`
- frame protections

Do not blindly add headers that break required functionality.

14. HTTPS and TLS

Verify HTTPS, HTTP-to-HTTPS redirects, Secure cookies where applicable, appropriate HSTS, no mixed content, and HTTPS API calls.

15. Cookie Security

Audit `Secure`, `HttpOnly`, `SameSite`, `Domain`, `Path`, and expiration. Sensitive cookies should not be JavaScript-readable unless strictly necessary.

16. File Upload Security

For trade screenshots and other uploads, verify:

- allowed file types
- MIME and extension validation
- file-size limits
- filename sanitization
- safe storage location
- executable-file prevention
- path-traversal prevention
- access control
- controlled public exposure

Prefer non-executable storage outside the public web root or controlled delivery.

17. Trade Screenshot Privacy

Verify User B cannot access User A's uploaded MT4/MT5 screenshots by guessing filenames, IDs, URLs, or attachment IDs. Authorization must be enforced by the backend.

18. Path Traversal

Check file endpoints against `../`, `..\`, encoded/double-encoded traversal, and absolute paths. User-controlled paths must not access server files.

19. SSRF

Where the application fetches external URLs, audit URL import, image fetching, webhook tests, external APIs, and AI integrations. User input must not reach private/internal network resources.

20. Webhook Security

Verify signature validation, replay protection, timestamp validation where applicable, secret management, and request authentication. Never trust a webhook based only on its URL.

21. Rate Limiting

Check login, registration, password reset, AI requests, uploads, trade creation/deletion, and API endpoints. Prevent abuse and excessive provider costs.

22. AI Security

Verify:

- API keys remain server-side
- uploaded images remain protected
- prompts cannot retrieve another user's data
- AI cannot execute arbitrary server commands
- AI output is treated as untrusted
- generated HTML is sanitized
- generated database queries are never executed blindly

23. Payment Security

If payment/subscription functionality exists, audit credentials, webhook signatures, server-side prices, subscriptions, entitlements, and payment-status verification. Never trust frontend payment status.

24. Error Handling

Production errors must not reveal paths, stack traces, queries, SQL errors, environment variables, framework internals, server configuration, or source code. Public errors should be generic.

25. Debug Mode

Verify production does not expose debug mode, verbose errors, development endpoints/configuration/credentials, or test routes.

26. Database Security

Check credentials, exposure, least privilege, remote access, backups, sensitive fields, password storage, and encryption where appropriate.

27. Backups

Check whether backups are public, predictable, downloadable, or contain secrets/user data. Backups must not reside in public directories.

28. Server and cPanel Security

Audit PHP configuration, file/directory permissions, `public_html` structure, dangerous functions where appropriate, server information, cron jobs, logs, temporary files, and backup directories.

Do not change server configuration during the audit.

29. Dependencies

Audit npm, Composer, PHP, and JavaScript dependencies for known vulnerabilities, outdated/abandoned packages, and unnecessary dependencies.

Do not automatically update dependencies; report first.

30. Frontend Security

Check whether browser-delivered code exposes API keys, internal/admin endpoints, secrets, private configuration, sensitive IDs, or debug information. Anything delivered to the browser is public.

31. Admin Security

Verify authentication, authorization, role checks, route/API protection, direct URL access, and privilege escalation. Hidden URLs are not an access-control mechanism.

32. Information Disclosure

Check disclosure of server/PHP/framework/database versions, internal paths, stack traces, debug information, user/internal object IDs, and unnecessary API metadata.

33. HTTP Method Security

Verify endpoints correctly restrict `GET`, `POST`, `PUT`, `PATCH`, and `DELETE`. Changing the method must not bypass authorization.

34. Security Regression Test

Compare the current version with the previous version for newly exposed files, broken authorization, API/upload/authentication weaknesses, sensitive-data leakage, dependency issues, CORS problems, and security-header regressions.

35. Security Severity

Classify every finding realistically as:

- `CRITICAL`
- `HIGH`
- `MEDIUM`
- `LOW`
- `INFO`

Do not inflate severity.

36. Safe Testing Rules

All testing must be non-destructive.

Do not:

- delete production data
- modify user accounts or passwords
- alter payments
- execute destructive SQL
- upload executable malware

- stress the server or run DoS tests
- expose private user information

37. Required Test Results

For every important test report:

- `TEST`
- `URL / COMPONENT`
- `EXPECTED RESULT`
- `ACTUAL RESULT`
- `STATUS`
- `SEVERITY`
- `RECOMMENDATION`

Example:

```
TEST:
Public .env access

URL:
/api/.env

EXPECTED:
404/403

ACTUAL:
404 JSON response

STATUS:
PASS

SEVERITY:
INFO
```

38. Release Security Gate

At the end calculate:

```
CRITICAL: 0
HIGH: 0
MEDIUM: X
LOW: X
INFO: X
```

Release decision:

- `SAFE TO RELEASE`
- `DO NOT RELEASE`

If any unresolved `CRITICAL` or `HIGH` vulnerability remains, do not recommend production release.

39. Final Report Format

```
# VELORA SECURITY AUDIT

Version:
[version]

Date:
[date]

Overall Status:
PASS / WARNING / FAIL

## 1. Critical Findings
## 2. High Findings
## 3. Medium Findings
## 4. Low Findings
## 5. Information Findings
## 6. Sensitive File Exposure
## 7. Authentication
## 8. Authorization
## 9. API Security
## 10. Database Security
## 11. File Upload Security
## 12. AI Security
## 13. Payment Security
## 14. HTTPS / TLS
## 15. Security Headers
## 16. Cookies
## 17. CORS
## 18. Error Disclosure
## 19. Dependencies
## 20. Server / cPanel
## 21. Security Regression
## 22. Tests Passed
## 23. Tests Failed
## 24. Recommended Fixes
## 25. Release Decision
```

Permanent Rule

This checklist must be reused for every Velora version.

Before every release:

```
RUN SECURITY AUDIT
↓
COMPARE WITH PREVIOUS VERSION
↓
IDENTIFY NEW RISKS
↓
IDENTIFY REGRESSIONS
↓
REPORT FINDINGS
↓
FIX APPROVED ISSUES
↓
RUN AUDIT AGAIN
↓
FINAL RELEASE DECISION
```

Never assume that a new version is secure because the previous version was secure.

Every version must pass the audit again.

Audit first. Report first. Ask for approval before making security changes. Do not automatically modify the project.